

Music Playlist app using FullStack

An Internship report submitted in partial fulfilment of the
requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

submitted by

Teki SaiRaj

22NR1A05D8

Under the Esteemed Guidance of

Ms. S. Harshini



(Estd: 2008)

BABA INSTITUTE OF TECHNOLOGY AND SCIENCES

(Autonomous)

(Approved by AICTE New Delhi, Affiliated to JNTU GV, ISO 9001:2015)

Certified, Accredited by NAAC with 'A' Grade)

Bakkannapalem Village, Madhurawada Post, Visakhapatnam – 530 048

2026

BABA INSTITUTE OF TECHNOLOGY AND SCIENCES

(Autonomous)

(Approved by AICTE New Delhi, Affiliated to JNTU GV, ISO 9001:2015

Certified, Accredited by NAAC with 'A' Grade)

Bakkannapalem Village, Madhurawada Post, Visakhapatnam – 530 048

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that this is the Internship report entitled “**Music Playlist app using FullStack**” is the bonafide work being submitted by **Teki SaiRaj, 22NR1A05D8** of final year in partial fulfillment of the requirement for the award of degree in Bachelor of Technology in Computer Science and Engineering in BITS (Approved by AICTE, affiliated to JNTUGV and accredited by NAAC). It is the record of bonafide work carried out under the esteemed guidance of Ms. S. Harshini, Dept. of CSE during the academic year 2025-2026.

Signature of Project Guide

Signature of HOD

Signature of External Examiner



DECLARATION

I **Teki SaiRaj, 22NR1A05D8** of final semester B.Tech, in the department of Computer Science and Engineering from **BABA INSTITUTE OF TECHNOLOGY AND SCIENCES, Visakhapatnam** hereby declare that the Internship Report entitled **Music Playlist app using FullStack** is carried out by us and submitted in partial fulfilment of the requirements for the award of **Bachelor of Technology in Computer Science and Engineering**, Under the guidance of, Ms. S. Harshini in **BABA INSTITUTE OF TECHNOLOGY AND SCIENCES** during the academic year 2021-2025 and has not been submitted to any other university for the award of any kind of degree.

Teki SaiRaj, 22NR1A05D8

ACKNOWLEDGEMENT

We would like to express our special thanks to Ms. S. Harshini, Assistant Professor, Department of Computer Science and Engineering, and our project guide who gave us moral support and guided us in different matters regarding the topic. He has been very kind and patient while suggesting to us the outlines of this Internship project, correcting our mistakes and clearing our doubts which helped us to complete our project successfully. We thank for his overall support.

We express our sincere and whole hearted gratitude to Associate Professor **MR. S. DURGA PRASAD**, Head of the Department, Computer Science and Engineering who gave us complete support for the fulfillment of our internship project.

We express our Grateful thanks to Principal **DR. M. RAJAN BABU**, Baba Institute of Technology and Sciences (BITS), for the overall valuable support he provided during the project work and also during our course of study.

We would like to express our special thanks and gratitude to the management of our esteemed institute “**BABA INSTITUTE OF TECHNOLOGY AND SCIENCES**”, who gave us the excellent opportunity to do this project and helped us in doing a lot of research and we came to know about so many things.

We would also like to thank our friends, teaching and non-teaching staff of the Department of Computer Science and Engineering for all their support at critical stages throughout our study. I would like to express my sincere thanks to our parents who encouraged us throughout our educational endeavor and do this project work. Finally, I would like to express my thanks to the library staff, who have supported me in the accomplishment of the Internship.

Teki SaiRaj, 22NR1A05D8

Abstract

This internship report documents the development of a comprehensive Music Playlist Application utilizing the MERN (MongoDB, Express.js, React.js, Node.js) stack. The project aims to address the growing need for personalized music management systems that enable users to create, organize, and interact with digital music collections in an intuitive and efficient manner.

The application provides a robust platform where users can perform essential music management operations including playlist creation, song organization, and audio playback functionality. The system architecture implements a full-stack JavaScript solution, with a RESTful API backend built on Node.js and Express.js, a MongoDB database for persistent data storage, and a dynamic React-based frontend for seamless user interaction.

Key technical implementations include the integration of GridFS for efficient audio file storage and streaming, Multer middleware for handling file uploads, and Axios for client-server communication. The application features collaborative playlist capabilities, allowing multiple users to contribute to shared collections, alongside comprehensive search and filtering functionalities for songs based on various parameters such as title, artist, and album.

The project successfully achieves its objective of delivering a functional music playlist application while providing valuable insights into full-stack development methodologies, database design considerations, and the implementation of audio streaming capabilities in web applications.

INDEX

Chapter No.	Title	Page No.
	Certificate	i
	Declaration	ii
	Acknowledgements	iii
	Abstract	iv
	List of Figures	v
1.	Introduction	1
1.1	Problem Statement	1
1.2	Objectives of the Project	3
1.3	Scope of the Project	4
2.	Requirements Analysis	6
2.1	Functional Requirements	6
2.1.1	User Registration and Authentication	6
2.1.2	User Profile Management	6
2.1.3	Music Upload and Management	6
2.1.4	Playlist Creation and Management	6
2.1.5	Music Streaming and Playback	7
2.1.6	Search and Filtering	7
2.1.7	Collaborative Features	7

Chapter No.	Title	Page No.
2.1.8	Notifications and Alerts	7
2.2	Non-Functional Requirements	7
2.2.1	Performance	7
2.2.2	Scalability	7
2.2.3	Usability	8
2.2.4	Security	8
2.2.5	Reliability	8
2.2.6	Compatibility	8
2.2.7	Maintainability	8
2.2.8	Availability	8
3.	Analysis and Design	9
3.1	Introduction to System Analysis and Design	9
3.2	System Architecture of the Music Playlist Application	10
3.3	Use Case Diagram	11
3.4	Sequence Diagram	13
3.5	Activity Diagram	15
3.6	Database Design	17
3.6.1	Database Architecture	17
3.6.2	Users Collection	17

Chapter No.	Title	Page No.
3.6.3	Songs Collection	19
3.6.4	Playlists Collection	20
3.6.5	Collaborators Collection	21
3.6.6	Playback History Collection	22
3.6.7	Relationships Between Collections	22
3.6.8	Indexing and Performance Optimization	23
3.6.9	Data Security	23
4.	Technology Stack	24
4.1	Overview of MERN Stack	24
4.2	MongoDB	25
4.2.1	NoSQL Database Concepts	25
4.2.2	GridFS for File Storage	27
4.3	Express.js Framework	28
4.3.1	REST API Development	28
4.3.2	Middleware Implementation	30
4.4	React.js Library	31
4.4.1	Component Architecture	31
4.4.2	State Management with Hooks	33
4.5	Node.js Runtime	34

Chapter No.	Title	Page No.
4.5.1	Event-Driven Architecture	34
4.5.2	Package Management with npm	35
4.6	Supporting Technologies	36
4.6.1	Multer for File Uploads	36
4.6.2	Axios for HTTP Requests	37
4.6.3	Mongoose ODM	38
4.6.4	CORS and Environment Variables	39
5.	Implementation Screenshots	40
5.1	Application User Interface	40
5.2	Running Application in Browser	41
5.3	Backend Project Structure	42
5.4	Frontend Project Structure	43
5.5	MongoDB Database Implementation	45
5.6	Playlist Creation Functionality	45
5.7	Song Playback Feature	46
5.8	Integration of Frontend and Backend	47
6.	Conclusion	48
References		49

Chapter 1: Introduction

1.1 Problem Statement

In the contemporary digital ecosystem, music enthusiasts face several challenges when attempting to manage their personal music collections effectively. While commercial streaming services offer comprehensive solutions, they come with recurring subscription costs, internet dependency, and limitations on user control over their music libraries. Conversely, traditional media players installed on personal computers often lack modern features such as cloud synchronization, collaborative playlist creation, and intuitive webbased interfaces accessible from any device.

The specific problems addressed by this Music Playlist Application can be categorized into several key areas:

Fragmented Music Collections: Users typically store music across multiple devices—smartphones, laptops, tablets, and external drives—resulting in fragmented libraries that are difficult to consolidate and manage centrally. This fragmentation leads to duplicate files, inconsistent metadata, and the inability to access the complete collection from any single device.

Limited Organization Capabilities: Traditional file explorers and basic media players provide rudimentary organization based on folder structures or simple metadata fields. Users lack sophisticated tools for creating hierarchical playlists, tagging songs with custom attributes, or implementing advanced search and filtering mechanisms that go beyond basic filename matching.

Collaborative Music Curation: Modern music consumption is often a social activity. Friends, family members, or colleagues may wish to contribute to shared playlists for events, road trips, or collaborative work environments. Existing solutions either lack collaborative features entirely or implement them in ways that are complex to configure and use.

Audio File Management Complexity: Handling audio files presents unique technical challenges. Files must be stored efficiently, streamed without buffering interruptions, and protected against unauthorized access. Metadata extraction, format compatibility, and playback synchronization require careful consideration in application design.

Cross-Platform Accessibility: Users expect to access their music collections from any device with internet connectivity. Desktop applications tied to specific operating systems fail to meet this expectation, while web-based solutions must balance functionality with performance across varying network conditions and device capabilities.

Technical Knowledge Barriers: Many existing solutions require users to possess technical knowledge about file formats, codecs, and server configurations. A well-designed application should abstract these complexities, presenting an intuitive interface that hides underlying technical implementation details.

The Music Playlist Application developed during this internship directly addresses these problems by providing a comprehensive, web-based solution that leverages modern fullstack technologies. Users can upload audio files, organize them into customizable playlists, search their collections using multiple criteria, and stream music directly through their web browsers. Collaborative features enable multiple users to contribute to shared playlists, fostering social music experiences. The application is accessible from any device with a modern web browser, eliminating platform dependencies and ensuring consistent user experiences.

From a technical perspective, the project addresses challenges related to efficient file storage and retrieval, real-time data synchronization, responsive user interface design, and scalable backend architecture. The implementation must handle concurrent user requests, manage audio streaming with minimal latency, and maintain data integrity across database operations.

1.2 Objectives of the Project

The primary objectives guiding the development of this Music Playlist Application are structured to ensure comprehensive coverage of both functional requirements and learning outcomes. These objectives serve as benchmarks against which the success of the project can be measured.

Primary Objectives:

1. **Develop a Fully Functional Music Playlist Application:** Create a working web application that enables users to perform core music management operations including uploading audio files, creating and modifying playlists, searching for songs, and streaming audio content. The application must demonstrate reliability, responsiveness, and adherence to specified requirements.
2. **Implement Full-Stack JavaScript Solution:** Utilize the MERN stack comprehensively to build both backend and frontend components. This includes designing MongoDB schemas, implementing Express.js routes and middleware, developing React components with state management, and configuring Node.js server environments.
3. **Design Intuitive User Interface:** Create a clean, accessible, and visually appealing user interface that facilitates easy navigation and operation. The interface should follow modern design principles, provide clear feedback for user actions, and maintain consistency across different sections of the application.
4. **Enable Audio File Management:** Implement robust mechanisms for uploading, storing, and retrieving audio files. This includes utilizing GridFS for efficient storage of large binary files, implementing proper streaming protocols for smooth playback, and managing file metadata appropriately.
5. **Provide Comprehensive Playlist Features:** Support creation, modification, deletion, and viewing of playlists. Each playlist should maintain references to

constituent songs, display relevant metadata, and support collaborative features where multiple users can contribute.

6. **Implement Search and Filtering Capabilities:** Enable users to search their music collections using various criteria including song titles, artist names, and album information. The search functionality should be efficient, responsive, and support partial matching for enhanced usability.
7. **Ensure Cross-Browser Compatibility:** Verify that the application functions correctly across major web browsers including Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. Responsive design principles should ensure usability on both desktop and mobile devices.
8. **Document Development Process:** Maintain comprehensive documentation throughout the development lifecycle, including requirements specifications, design decisions, implementation details, and testing procedures. This documentation serves both as a learning record and as reference material for future enhancements.
9. **Apply Industry Best Practices:** Follow established conventions for code organization, version control, error handling, and security implementation. The codebase should be maintainable, readable, and structured according to professional standards.
10. **Demonstrate Technical Competence:** Showcase proficiency in full-stack development concepts including REST API design, database modeling, state management, asynchronous programming, and middleware implementation.

1.3 Scope of the Project

The scope of this Music Playlist Application defines the boundaries of what the project encompasses and, equally importantly, what lies outside its intended coverage. Clear

scope definition ensures focused development efforts and sets appropriate expectations for project deliverables.

Functional Coverage: The application provides complete functionality for creating and managing playlists, uploading and organizing songs, and streaming audio content. Users can create multiple playlists, add songs to playlists, view playlist contents, and remove songs from playlists. Audio playback includes play, pause, and track selection capabilities with visual indicators for currently playing content.

User Management: Basic user identification is supported through simple user ID parameters, enabling collaborative playlist features. However, comprehensive authentication systems including user registration, login, password management, and session handling are considered beyond the current scope.

Audio Format Support: The application supports common audio formats suitable for web streaming, with particular attention to formats that enable efficient browser-based playback without requiring additional plugins or codecs.

Database Operations: All data persistence operations including CRUD (Create, Read, Update, Delete) functions for playlists and songs are fully implemented. Database design follows normalization principles appropriate for NoSQL document stores while maintaining data integrity.

API Development: A complete RESTful API is implemented to support all frontend operations. API endpoints follow consistent naming conventions, return appropriate HTTP status codes, and include error handling for various failure scenarios.

Frontend Interface: The user interface includes all screens necessary for application functionality: main dashboard showing playlists and songs, playlist creation forms, song listing views, and audio player controls. CSS styling ensures visual appeal and usability.

Requirements of the System

1. Functional Requirements

Functional requirements describe the specific features and operations that the system must perform to satisfy the user needs described in the problem statement.

1.1 User Registration and Authentication

The system shall allow users to create a new account using an email address and password.

The system shall provide login functionality so registered users can access their accounts securely.

The system shall allow users to log out of the system after using the application. The system shall maintain secure session management to protect user data and account access.

1.2 User Profile Management

The system shall allow users to view their profile information.

The system shall allow users to update profile details such as username or password.

The system shall store user-specific information including playlists and music preferences.

1.3 Music Upload and Management

The system shall allow users to upload audio files from their local devices.

The system shall support common audio formats such as MP3, WAV, and AAC.

The system shall store uploaded songs in a structured database.

The system shall allow users to edit song metadata such as title, artist, and album name.

The system shall allow users to delete songs from their personal music collection.

1.4 Playlist Creation and Management

The system shall allow users to create new playlists.

The system shall allow users to add songs to playlists.

The system shall allow users to remove songs from playlists.

The system shall allow users to rename playlists.

The system shall allow users to delete playlists when they are no longer needed.

The system shall allow users to reorder songs within playlists.

1.5 Music Streaming and Playback

The system shall allow users to stream music directly from the application.

The system shall provide playback controls such as play, pause, next, and previous.

The system shall allow users to play songs sequentially or shuffle the playlist.

The system shall display the currently playing song.

1.6 Search and Filtering

The system shall allow users to search songs by title, artist, or album.

The system shall provide quick filtering of songs within the music library.

The system shall allow users to browse songs using playlists or categories.

1.7 Collaborative Features

The system shall allow users to share playlists with other users.

The system shall allow multiple users to collaborate on playlists.

The system shall allow users to access shared playlists.

1.8 Notifications and Alerts

The system shall display notifications when songs are uploaded successfully.

The system shall display alerts when errors occur during operations.

2. Non-Functional Requirements

Non-functional requirements describe the quality attributes and performance standards expected from the system.

2.1 Performance

The system should load music libraries and playlists quickly.

The system should support smooth audio streaming with minimal buffering.

Search operations should return results quickly even for large music collections.

2.2 Scalability

The system should be capable of handling an increasing number of users.

The system should support large music libraries without affecting performance.

2.3 Usability

The system should provide a simple and user-friendly interface.

Users should be able to perform tasks such as uploading songs and creating playlists easily.

The application should be easy to use for both technical and non-technical users.

2.4 Security

User passwords should be securely encrypted before storage.

The system should implement authentication and authorization mechanisms.

Only authorized users should be able to access their personal music collections.

2.5 Reliability

The system should operate consistently without frequent failures.

The system should ensure that user data and music files are stored safely.

2.6 Compatibility

The application should work on modern web browsers such as Chrome, Firefox, and Edge.

The system should function across different devices including desktops, laptops, and mobile browsers.

2.7 Maintainability

The system should follow a modular design so that updates and improvements can be implemented easily.

The code should be well organized and documented for future maintenance.

2.8 Availability

The system should be available for users at all times.

The system should ensure minimal downtime during updates or maintenance.

3. Analysis and Design

3.1 Introduction to System Analysis and Design

System analysis and design is an essential phase in the software development life cycle (SDLC). In this phase, the system requirements gathered during the requirement analysis phase are transformed into a structured system architecture and design model. The main objective of this phase is to clearly define how the system will function, how the data will flow between different components, and how the system will interact with users.

Analysis focuses on understanding the functional requirements and identifying the major processes that the system must perform. Design focuses on creating a blueprint for the implementation of the system, including the system architecture, database design, and the interaction between different modules.

In this project, a Music Playlist Application is developed using the MERN stack, which includes MongoDB, Express.js, React.js, and Node.js. The application enables users to upload songs, create playlists, search songs, play music, and collaborate with other users. To clearly represent the behavior and structure of the system, several UML diagrams are used.

UML (Unified Modeling Language) diagrams provide a visual representation of the system. They help developers and stakeholders understand how the system works before actual implementation begins.

The following UML diagrams are used in this project:

- Use Case Diagram
- Sequence Diagram
- Activity Diagram

These diagrams help in analyzing the system from different perspectives. The use case diagram explains system functionality, the sequence diagram shows interactions between system components, and the activity diagram describes the workflow of the system.

3.2 System Architecture of the Music Playlist Application

The Music Playlist Application follows a three-tier architecture, which separates the system into three layers:

1. Presentation Layer
2. Application Layer
3. Data Layer

This architecture improves scalability, maintainability, and performance of the system.

Presentation Layer

The presentation layer is responsible for the user interface. It allows users to interact with the system through a web browser. In this project, the presentation layer is developed using React.js.

React.js provides a dynamic and responsive interface that allows users to easily perform operations such as:

- Uploading songs
- Creating playlists
- Playing music
- Searching songs

The frontend communicates with the backend through REST API requests.

Application Layer

The application layer handles the business logic of the system. It processes user requests and performs operations on the database.

In this project, the application layer is implemented using Node.js and Express.js. Node.js provides the runtime environment for executing server-side JavaScript, while Express.js simplifies the development of web APIs.

The backend performs tasks such as:

- Processing user authentication
- Managing playlists

- Handling song uploads
- Fetching song data from the database
- Managing collaboration between users

Data Layer

The data layer stores all the information required by the application. MongoDB is used as the database system.

MongoDB stores data in JSON-like documents, making it flexible and suitable for web applications.

The database stores information such as:

- User accounts
- Songs
- Playlists
- Collaborative playlist data

3.3 Use Case Diagram

A Use Case Diagram represents the interactions between users and the system. It identifies the actors involved in the system and the functionalities that they can perform.

The main actors in the Music Playlist Application are:

User

Collaborator

The user is the primary actor who interacts with the system to manage music and playlists. The collaborator is a secondary actor who can access and contribute to shared playlists.

Use Cases in the System

View Playlists

This functionality allows users to view all the playlists available in their account.

Users can browse their playlists and select one to play songs.

Create Playlist

Users can create a new playlist by providing a name and selecting songs from the music library.

Generate Playlist Code

The system generates a unique playlist code when a playlist is created. This code can be shared with collaborators so they can access the playlist.

Upload Song

Users can upload songs from their local device to the music library.

Play Song

This feature allows users to stream and listen to songs directly in the application.

Control Playback

Users can control music playback using options such as play, pause, next, and previous.

Search Songs

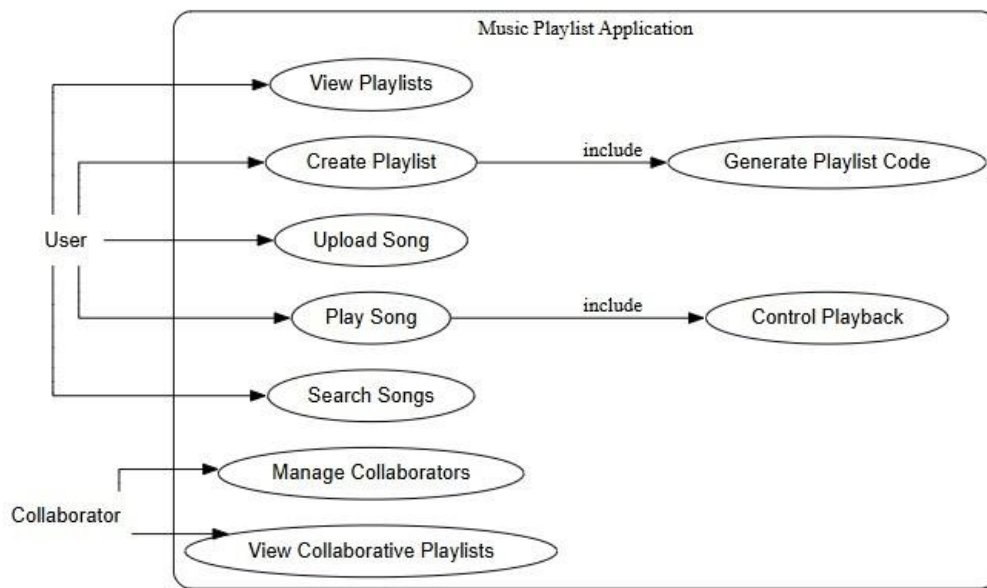
Users can search songs using keywords such as song title, artist name, or album name.

Manage Collaborators

Users can add or remove collaborators who are allowed to modify shared playlists.

View Collaborative Playlists

Collaborators can view playlists that have been shared with them.



Importance of the Use Case Diagram

The use case diagram helps in identifying the system functionalities and defining the interactions between the actors and the system. It provides a clear understanding of how users will interact with the application and what services the system provides.

3.4 Sequence Diagram

A Sequence Diagram shows the sequence of interactions between different components of the system over time. It describes how objects communicate with each other to complete a particular process.

In the Music Playlist Application, the sequence diagram represents the process of creating a playlist.

Components Involved

User

Browser

PlaylistController

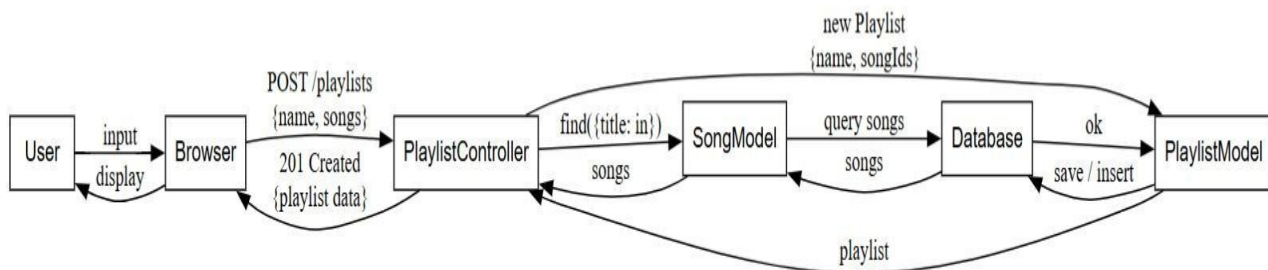
SongModel

PlaylistModel

Database

Sequence of Events

1. The user interacts with the browser interface to create a playlist.
2. The browser sends a POST request to the PlaylistController.
3. The PlaylistController receives the request and validates the input.
4. The controller sends a request to the SongModel to retrieve song information.
5. The SongModel queries the database for song records.
6. The database returns the song data.
7. The PlaylistController creates a new playlist object.
8. The PlaylistModel stores the playlist data in the database.
9. The database confirms successful insertion.
10. The browser receives a success response.



Importance of Sequence Diagram

The sequence diagram helps developers understand the order of operations and the communication between system components. It ensures that each module interacts properly with other modules in the system.

3.5 Activity Diagram

An Activity Diagram represents the workflow of the system. It shows the sequence of activities that occur during system operation.

In the Music Playlist Application, the activity diagram describes the steps involved when a user interacts with the system.

Workflow Description

1. The user opens the application.
2. The React application loads in the browser.
3. The system fetches songs and playlists from the database.
4. The dashboard is displayed to the user.
5. The user chooses an action.

Possible actions include:

- View Playlists
- Create Playlist
- Search Songs

If the user selects Create Playlist:

The user enters the playlist name and selects songs.

The system validates the input.

The playlist is stored in the database.

The system refreshes the dashboard.

If the user selects Search Songs:

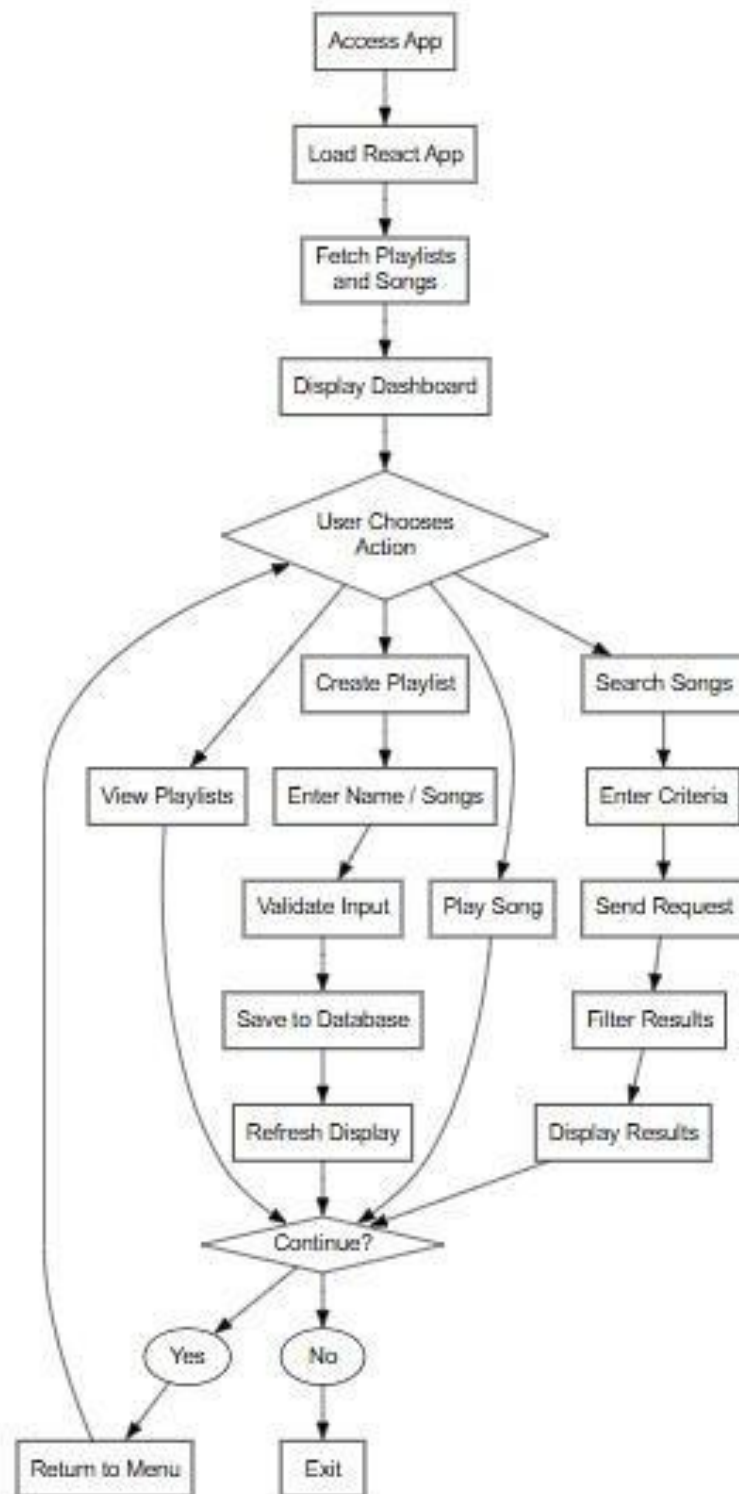
The user enters search criteria.

The system sends a request to the server.

The server filters songs from the database.

Search results are displayed to the user.

Finally, the user decides whether to continue using the application or exit.



Importance of Activity Diagram

The activity diagram helps developers understand the workflow of the system and identify possible decision points in the application.

3.6 Database Design

Database design is one of the most important stages in system design because it determines how data is stored, managed, and retrieved efficiently. A well-structured database ensures that the system performs efficiently, maintains data integrity, and supports scalability.

In the Music Playlist Application, the database is used to store user information, songs, playlists, and collaboration data. The system uses MongoDB, which is a document-oriented NoSQL database. Unlike traditional relational databases that use tables and rows, MongoDB stores data in collections and documents using JSON-like structures.

The flexible schema design of MongoDB allows developers to store complex data structures easily, making it suitable for modern web applications such as music streaming platforms.

3.6.1 Database Architecture

The database architecture of the Music Playlist Application consists of several collections that store different types of data. Each collection is responsible for storing specific information required for the operation of the application.

The main collections used in the system include:

- Users Collection
- Songs Collection
- Playlists Collection
- Collaborators Collection
- Playback History Collection

Each collection is designed to store structured information that supports the functionality of the application.

3.6.2 Users Collection

The Users collection stores the information of all registered users of the application. Each user is uniquely identified by a User ID.

This collection helps manage authentication, authorization, and user-specific data such as playlists and uploaded songs.

Attributes of Users Collection

UserID

Unique identifier for each user.

Username

The display name of the user.

Email

The email address used for login and communication.

Password

Encrypted password stored securely in the database.

ProfileImage

Stores the path of the user profile image.

CreatedDate

Date when the user account was created.

Role

Defines whether the user is a normal user or an administrator.

Example Document Structure

```
{
  "_id": "U1001",
  "username": "SaiRaj",
  "email": "sairaj@example.com",
  "password": "encrypted_password",
  "profileImage": "profile1.png",
  "createdDate": "2026-02-10",
  "role": "user"
}
```

Importance of Users Collection

This collection ensures that each user has a unique account and allows the system to manage personalized playlists and music collections.

3.6.3 Songs Collection

The Songs collection stores metadata related to music files uploaded by users. Instead of storing the actual audio files in the database, the system stores file paths that point to the location where the songs are stored on the server.

This approach improves database performance and reduces storage overhead.

Attributes of Songs Collection

SongID

Unique identifier for each song.

Title

Name of the song.

Artist

Name of the artist who performed the song.

Album

Album name associated with the song.

Genre

Category of the music.

Duration

Length of the song.

FilePath

Location of the audio file stored on the server.

UploadedBy

User ID of the person who uploaded the song.

UploadDate

Date when the song was uploaded.

Example Document Structure

```
{  
  "_id": "S102",  
  "title": "Dream Melody",  
  "artist": "John Smith",  
  "album": "Summer Beats",  
  "genre": "Pop",  
  "duration": "3:45",  
  "filePath": "/uploads/song102.mp3",  
  "uploadedBy": "U1001",  
  "uploadDate": "2026-02-15"  
}
```

Importance of Songs Collection

This collection allows the system to efficiently store and retrieve songs, perform searches, and organize music based on various attributes.

3.6.4 Playlists Collection

The Playlists collection stores information about playlists created by users. Each playlist contains a list of songs and information about the user who created it.

Attributes of Playlists Collection

PlaylistID

Unique identifier for the playlist.

PlaylistName

Name of the playlist created by the user.

CreatedBy

User ID of the playlist creator.

SongIDs

Array containing IDs of songs included in the playlist.

Collaborators

List of users who can access or edit the playlist.

CreatedDate

Date when the playlist was created.

Visibility

Defines whether the playlist is public or private.

Example Document Structure

```
{
  "_id": "P501",
  "playlistName": "Workout Mix",
  "createdBy": "U1001",
  "songIds": ["S101", "S102", "S103"],
  "collaborators": ["U1002", "U1003"],
  "createdDate": "2026-02-20",
  "visibility": "private"
}
```

Importance of Playlists Collection

This collection helps users organize songs into meaningful groups. It also supports collaborative playlists where multiple users can contribute songs.

3.6.5 Collaborators Collection

The Collaborators collection manages access control for collaborative playlists. It defines which users can access or edit a playlist.

Attributes

CollaboratorID

Unique identifier for the collaborator entry.

PlaylistID

Playlist associated with the collaborator.

UserID

User who is granted access to the playlist.

PermissionLevel

Defines whether the user can view or edit the playlist.

Example Document

```
{
  "_id": "C201",
  "playlistId": "P501",
  "userId": "U1002",
  "permissionLevel": "edit"
}
```

3.6.6 Playback History Collection

This collection records the songs played by users. It helps in providing personalized recommendations and recently played lists.

Attributes

HistoryID

Unique identifier for the playback record.

UserID

User who played the song.

SongID

Song that was played.

PlayedAt

Timestamp when the song was played.

Example Document

```
{
  "_id": "H301",
  "userId": "U1001",
  "songId": "S102",
  "playedAt": "2026-03-10T10:15:00"
}
```

3.6.7 Relationships Between Collections

Although MongoDB is a NoSQL database, relationships between collections still exist.

The main relationships in the system include:

User → Playlist (One-to-Many)

One user can create multiple playlists.

Playlist → Songs (Many-to-Many)

A playlist can contain many songs, and a song can appear in multiple playlists.

Playlist → Collaborators (One-to-Many)

A playlist can have multiple collaborators.

User → Playback History (One-to-Many)

A user can have multiple playback records.

These relationships allow the system to manage complex interactions between users, songs, and playlists.

3.6.8 Indexing and Performance Optimization

To improve database performance, indexing is used on frequently searched fields such as:

Song Title

Artist Name

Playlist Name

User Email

Indexes allow the database to retrieve information quickly without scanning the entire collection.

3.6.9 Data Security

Data security is an important aspect of database design.

The following measures are implemented:

- Passwords are encrypted before storing in the database.
- Access control is applied to protect user data.
- Input validation prevents malicious data from being stored.
- Only authenticated users can perform database operations.

Chapter 4: Technology Stack

4.1 Overview of MERN Stack

The Music Playlist Application is built using the MERN stack, a powerful combination of four key technologies that enable full-stack JavaScript development. The acronym MERN stands for MongoDB, Express.js, React.js, and Node.js. This stack has gained immense popularity in modern web development due to its end-to-end JavaScript implementation, which streamlines development by allowing developers to use a single programming language across both client and server environments.

Table 4.1: Technology Stack Summary

Technology	Version	Purpose	Key Features Utilized
MongoDB	8.2.1	Database	Document storage, GridFS for files, Flexible schema
Express.js	4.18.3	Backend Framework	REST API routing, Middleware, Request handling
React.js	18.2.0	Frontend Library	Component architecture, Hooks, State management
Node.js	Latest LTS	Runtime Environment	Non-blocking I/O, npm ecosystem, Server operations

The MERN stack follows a classic three-tier architecture pattern. React.js handles the presentation tier, running in the browser and providing the user interface. Express.js and Node.js form the application tier, processing HTTP requests and implementing business logic. MongoDB serves as the data tier, persisting application data and audio files. This separation of concerns enables modular development, where each layer can be developed and maintained independently while communicating through well-defined interfaces.

The decision to use the MERN stack for this project was driven by several factors. First, the unified JavaScript language across all layers reduces context switching and accelerates development. Second, the vast ecosystem of npm packages provides solutions for common development challenges. Third, the stack's popularity ensures extensive documentation and community support, which proved invaluable during the learning process. Finally, the stack's alignment with modern web development practices provides valuable experience applicable to industry roles.

4.2 MongoDB

4.2.1 NoSQL Database Concepts

MongoDB is a document-oriented NoSQL database that stores data in flexible, JSON-like documents rather than the rigid tables used in traditional relational databases. This document model aligns naturally with the object-oriented programming paradigm and allows for intuitive data representation.

Key Characteristics:

- **Document-Oriented:** Data is stored in BSON (Binary JSON) format, allowing nested structures and arrays
- **Schema Flexibility:** Documents in the same collection can have different fields
- **Horizontal Scaling:** Built-in sharding capabilities for distributing data across servers
- **Rich Query Language:** Supports CRUD operations, aggregation pipelines, and geospatial queries

Comparison with Relational Databases:

Table 4.2: MongoDB vs. RDBMS Comparison

Aspect	MongoDB	Relational Database
Data Model	Document-oriented	Table-based

Aspect	MongoDB	Relational Database
Schema	Flexible, dynamic	Rigid, predefined
Relationships	Embedded or referenced	Foreign keys with joins
Query Language	JSON-based methods	SQL
Scalability	Horizontal (sharding)	Vertical (scaling up)
Use Case	Hierarchical data, rapid iteration	Complex transactions, strict con

In the Music Playlist Application, MongoDB's document model proves particularly advantageous. A playlist document naturally contains an array of song references, mirroring the conceptual relationship. Song documents embed all metadata directly, eliminating the need for joins when displaying song information. This document-oriented approach results in intuitive data structures and efficient query patterns.

4.2.2 GridFS for File Storage

GridFS is MongoDB's specification for storing and retrieving large files, such as audio files, that exceed the BSON document size limit of 16MB. GridFS divides files into chunks and stores each chunk as a separate document, enabling efficient storage and streaming of large binary data.

GridFS Architecture:

- **fs.files Collection:** Stores file metadata including filename, content type, and file size
- **fs.chunks Collection:** Stores binary chunks with sequence numbers for reassembly
- **Chunk Size:** Default 255KB, configurable based on application needs

Implementation in the Application:

javascript

```
const { GridFsStorage } = require('multer-gridfs-storage');
```

```
const { GridFSBucket } = require('mongodb');
```

```
// Create storage engine
```

```
const storage = new GridFsStorage({  
  url: 'mongodb://localhost:27017/musicdatabase',  
  file: (req, file) => {  
    return {  
      filename: file.originalname,  
      bucketName: 'uploads'  
    };  
  }  
});
```

```
// Initialize GridFS bucket for streaming
```

```
const gfs = new GridFSBucket(conn.db, {  
  bucketName: 'uploads'  
});
```

Advantages of GridFS for Audio Storage:

- Seamless integration with MongoDB, eliminating separate file storage systems
- Automatic chunking enables efficient streaming without loading entire files
- Metadata storage alongside file data simplifies file management

- Built-in replication and sharding for scalability

4.3 Express.js Framework

4.3.1 REST API Development

Express.js is a minimalist web framework for Node.js that provides a robust set of features for building web applications and APIs. It serves as the backbone of the backend, handling HTTP requests, routing, and middleware integration.

Core Features Used:

- **Routing:** Defining endpoints for different HTTP methods and URLs
- **Middleware:** Processing requests through a pipeline of functions
- **Request/Response Handling:** Parsing incoming data and formatting responses
- **Error Handling:** Centralized error management

Example Route Implementation:

javascript

```
// Get all playlists with populated songs
```

```
app.get('/api/playlists', async (req, res) => {  
  try {  
    const playlists = await Playlist.find().populate('songs');  
    res.json(playlists);  
  } catch (err) {  
    console.error(err);  
    res.status(500).json({ error: 'Server error' });  
  }  
});
```

```
// Create new playlist

app.post('/api/playlists', async (req, res) => {

  try {

    const { name, songs } = req.body;

    const existingSongs = await Song.find({ title: { $in: songs } });

    const songIds = existingSongs.map(song => song._id);

    const playlist = new Playlist({ name, songs: songIds });

    await playlist.save();

    res.json(playlist);

  } catch (err) {

    res.status(500).json({ error: 'Server error' });

  }

});
```

4.3.2 Middleware Implementation

Middleware functions are the building blocks of Express.js applications, executing during the request-response cycle. They can execute code, modify request/response objects, end the cycle, or call the next middleware.

Table 4.3: Express.js Middleware Functions

Middleware	Purpose	Implementation
CORS	Enable cross-origin requests	<code>app.use(cors())</code>
Body Parser	Parse JSON request bodies	<code>app.use(bodyParser.json())</code>

Middleware	Purpose	Implementation
Multer	Handle file uploads	upload.single('audioFile')
Error Handler	Centralized error processing	Custom error middleware
Static Files	Serve frontend assets	app.use(express.static('public'))

Custom Error Handling Middleware:

javascript

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send('Something went wrong!');  
});
```

The middleware architecture enables clean separation of concerns. Each middleware handles a specific aspect of request processing, making the code modular, testable, and maintainable.

4.4 React.js Library

4.4.1 Component Architecture

React.js is a JavaScript library for building user interfaces through reusable components. Its component-based architecture promotes code reusability, maintainability, and efficient rendering through the virtual DOM.

Component Hierarchy:

Table 4.4: React Components and Their Responsibilities

Component	Type	Responsibilities
App	Parent	Overall layout, state coordination

Component	Type	Responsibilities
PlaylistList	Container	Display playlists, hover effects, create form
SongList	Container	Display songs, playback controls
AudioPlayer	Presentational	Play/pause button, now playing indicator

Component Implementation Example:

javascript

```
function PlaylistList() {

  const [playlists, setPlaylists] = useState([]);

  const [hoveredPlaylist, setHoveredPlaylist] = useState(null);

  useEffect(() => {

    axios.get('http://localhost:5000/api/playlists')

      .then(response => setPlaylists(response.data))

      .catch(error => console.error('Error:', error));

  }, []);

  return (

    <div className="playlist-container">

      <h2 className="playlist-title">Playlists</h2>

      {playlists.map(playlist => (

        <div key={playlist._id}

          onMouseEnter={() => setHoveredPlaylist(playlist._id)}


```



```

      onMouseLeave={() => setHoveredPlaylist(null)}>
    <h3>{playlist.name}</h3>

    {hoveredPlaylist === playlist._id && (
      <ul>

        {playlist.songs?.map(song => (
          <li key={song._id}>{song.title}</li>

        ))}

      </ul>

    )}

  </div>

  )}

</div>

);
}

```

4.4.2 State Management with Hooks

React Hooks, introduced in React 16.8, enable state and lifecycle features in functional components. The Music Playlist App utilizes several built-in hooks:

useState: Manages component-level state

javascript

```
const [songs, setSongs] = useState([]);
```

```
const [currentSong, setCurrentSong] = useState(null);
```

```
const [audio, setAudio] = useState(null);
```

useEffect: Handles side effects like API calls

javascript

```
useEffect(() => {
```

```

// Fetch songs on component mount

axios.get('http://localhost:5000/api/songs')

  .then(response => setSongs(response.data));

}, []); // Empty dependency array = run once

Custom Logic: Playback control implementation

javascript

const playSong = (song) => {

  if (!currentSong || currentSong._id !== song._id) {

    if (audio) audio.pause();

    const newAudio = new
Audio(`http://localhost:5000/api/songs/${song._id}/audio`);

    setCurrentSong(song);

    setAudio(newAudio);

    newAudio.play();

  } else {

    audio.paused ? audio.play() : audio.pause();

  }

};

```

4.5 Node.js Runtime

4.5.1 Event-Driven Architecture

Node.js is a JavaScript runtime built on Chrome's V8 engine, designed for building scalable network applications. Its event-driven, non-blocking I/O model makes it particularly suitable for I/O-intensive applications like the Music Playlist App.

Key Characteristics:

- **Single-Threaded Event Loop:** Handles concurrent connections efficiently

- **Non-Blocking I/O:** Operations don't block the execution thread
- **Event Emitters:** Built-in pattern for handling asynchronous events

Event Loop in Audio Streaming:

When multiple users stream audio simultaneously, Node.js handles each request without creating new threads for each connection. The event loop registers callbacks for I/O operations (reading from GridFS) and continues processing other requests while data is being read. This architecture enables the server to handle numerous concurrent streams with minimal resource consumption.

4.5.2 Package Management with npm

npm (Node Package Manager) is the default package manager for Node.js, providing access to the largest ecosystem of open-source libraries. The Music Playlist App leverages several npm packages:

Table 4.5: npm Dependencies and Their Purpose

Package	Version	Purpose
express	4.18.3	Web framework
mongoose	8.2.1	MongoDB ODM
cors	2.8.5	Cross-origin resource sharing
multer	1.4.5	File upload handling
multer-gridfs-storage	5.0.2	GridFS storage engine
gridfs-stream	1.1.1	GridFS streaming
dotenv	16.4.5	Environment variable management

Package	Version	Purpose
shortid	2.2.16	Unique ID generation
axios	1.6.7	HTTP client (frontend)

Package.json Configuration:

```
json
```

```
{
  "dependencies": {
    "express": "^4.18.3",
    "mongoose": "^8.2.1",
    "cors": "^2.8.5",
    "multer": "^1.4.5-lts.1",
    "multer-gridfs-storage": "^5.0.2",
    "dotenv": "^16.4.5",
    "shortid": "^2.2.16"
  }
}
```

4.6 Supporting Technologies

4.6.1 Multer for File Uploads

Multer is a Node.js middleware for handling multipart/form-data, primarily used for uploading files. In the Music Playlist App, Multer integrates with GridFS storage to handle audio file uploads efficiently.

Configuration:

```
javascript
```

```

const multer = require('multer');

const { GridFsStorage } = require('multer-gridfs-storage');

const storage = new GridFsStorage({
  url: 'mongodb://localhost:27017/musicdatabase',
  file: (req, file) => {
    return {
      filename: file.originalname,
      bucketName: 'uploads'
    };
  }
});

const upload = multer({ storage });

// Upload endpoint
app.post('/api/upload', upload.single('audioFile'), (req, res) => {
  res.json({ fileId: req.file.id });
});

```

4.6.2 Axios for HTTP Requests

Axios is a promise-based HTTP client for the browser and Node.js. The frontend uses Axios to communicate with the backend API, handling requests, responses, and error conditions elegantly.

Usage Examples:

javascript

// GET request

```
axios.get('http://localhost:5000/api/songs')

  .then(response => setSongs(response.data))

  .catch(error => console.error('Error fetching songs:', error));
```

// POST request with data

```
axios.post('http://localhost:5000/api/playlists', {

  name: newPlaylistName,

  songs: newPlaylistSongs

});
```

4.6.3 Mongoose ODM

Mongoose is an Object Document Mapping (ODM) library for MongoDB and Node.js. It provides a schema-based solution for modeling application data, including built-in type casting, validation, and query building.

Schema Definitions:

javascript

```
const playlistSchema = new mongoose.Schema({

  name: { type: String, required: true },

  songs: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Song' }],

  playlistCode: { type: String, required: true, unique: true }

});

const songSchema = new mongoose.Schema({

  title: { type: String, required: true },

  artist: { type: String, required: true },
```

```

    duration: { type: Number, required: true },

    fileId: { type: mongoose.Schema.Types.ObjectId, ref: 'uploads.files' }

  });

```

Population for Relationships:

```

javascript

// Automatically replace song IDs with actual song documents

const playlists = await Playlist.find().populate('songs');

```

4.6.4 CORS and Environment Variables

CORS (Cross-Origin Resource Sharing):

CORS is a security mechanism that controls which origins can access server resources. The backend configures CORS to accept requests only from the frontend origin:

```

javascript

const cors = require('cors');

app.use(cors()); // In development, allows all origins

// For production, restrict to specific origin:

// app.use(cors({ origin: 'https://yourdomain.com' }));

```

Environment Variables:

Environment variables separate configuration from code, enabling different settings across development and production environments:

```

javascript

require('dotenv').config();

const PORT = process.env.PORT || 5000;

const MONGODB_URI = process.env.MONGODB_URI ||
'mongodb://localhost:27017/musicdatabase';

mongoose.connect(MONGODB_URI);

```

5. Implementation Screenshots

5.1 Application User Interface

The Music Playlist Application provides a simple and user-friendly interface that allows users to interact with the system efficiently. The user interface is developed using React.js and CSS to create a responsive and visually appealing layout.

The main interface consists of two primary sections:

- Playlist management section
- Song library section

Users can view playlists, create new playlists, and play songs directly from the application dashboard.

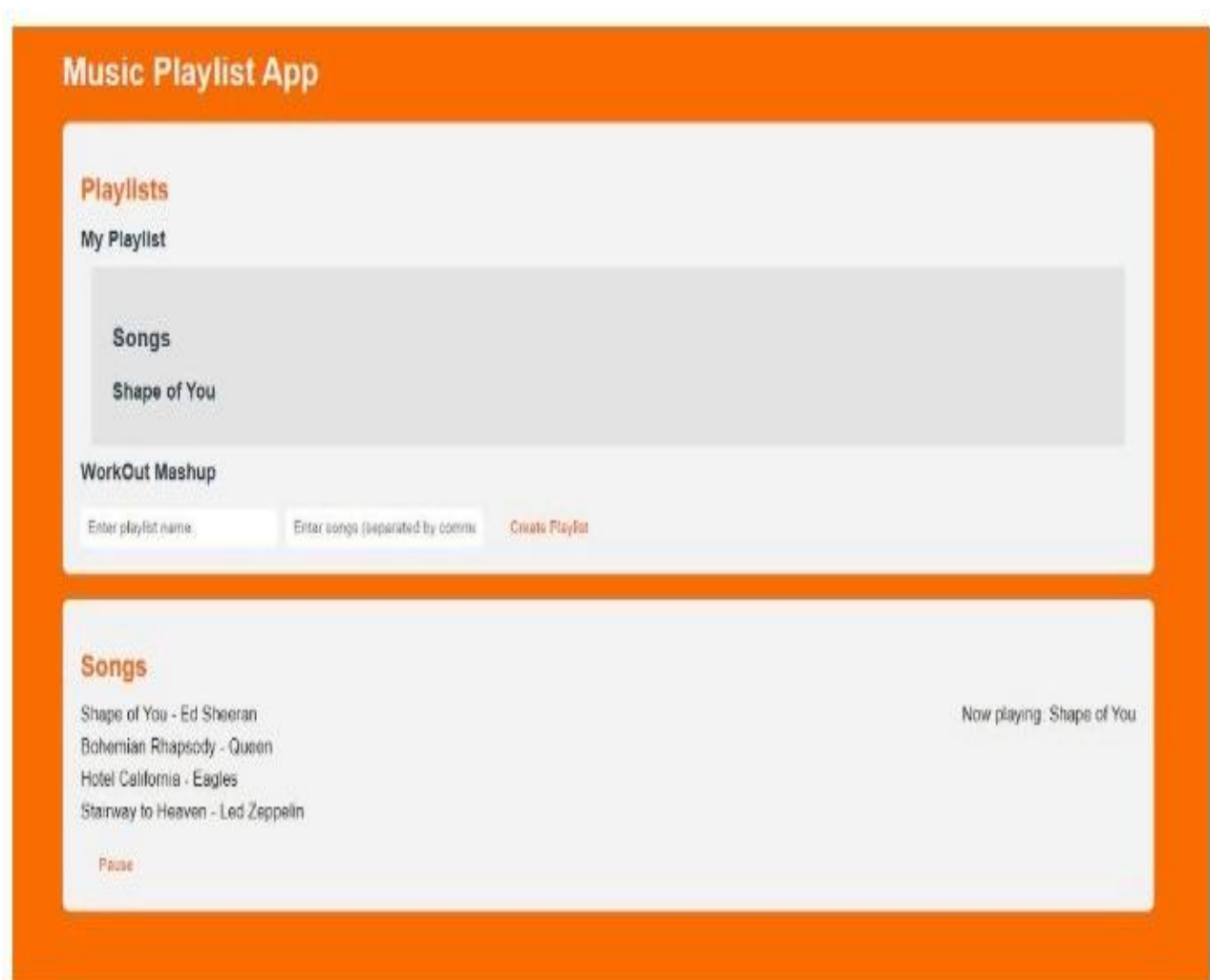


Figure 5.1: Main User Interface of the Music Playlist Application.

5.2 Running Application in Browser

After starting the frontend server, the application runs in the browser at **localhost:3000**. The dashboard displays available playlists and songs in the music library.

Users can perform actions such as:

- Creating playlists
- Selecting songs
- Playing songs
- Managing playlists

This interface allows users to interact with the backend server and retrieve song information stored in the database.

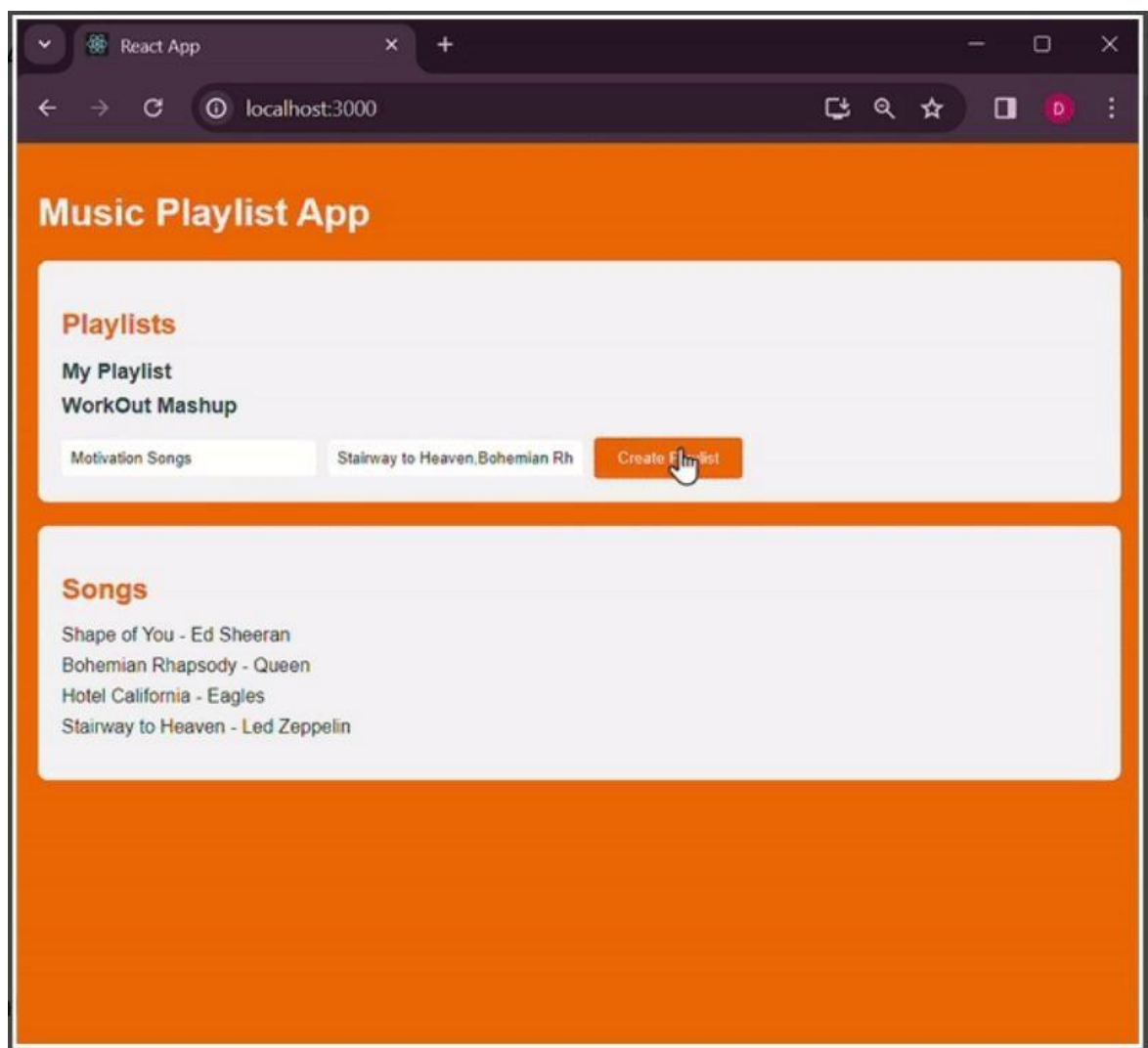


Figure 5.2: Running React Application in Web Browser.

5.3 Backend Project Structure

The backend of the Music Playlist Application is developed using Node.js and Express.js. The backend project contains important configuration and server files required to run the application.

The backend directory structure includes:

node_modules

Contains installed dependencies.

package.json

Stores project metadata and dependency information.

package-lock.json

Locks the dependency versions used in the project.

server.js

The main entry point of the backend application that starts the server and handles API routes. This structure helps maintain organization and ensures the backend server runs correctly.

Project Structure(Backend):

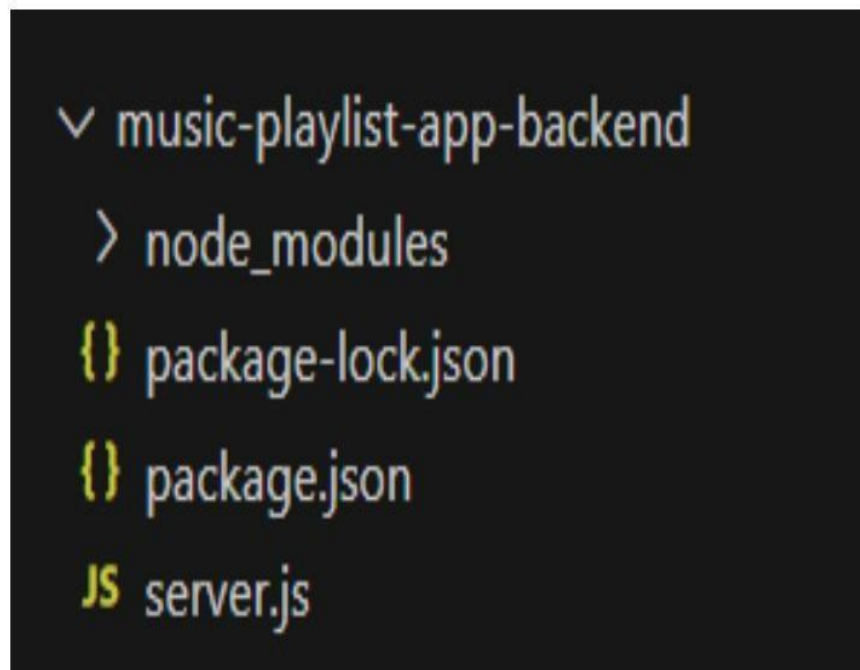


Figure 5.3: Backend Project Folder Structure.

5.4 Frontend Project Structure

The frontend of the Music Playlist Application is developed using React. React allows developers to create dynamic and responsive user interfaces by using reusable components. The frontend project follows a modular structure that organizes different parts of the application into separate files, making the code easier to manage and maintain. Important files in the frontend project include:

src folder

This folder contains the main source code of the React application. All components, stylesheets, and JavaScript files used to build the user interface are stored inside this folder.

App.js

App.js is the main React component that controls the layout of the application. It renders other components such as Playlist and Songs and manages the overall structure of the user interface.

Playlist.js

This component is responsible for handling playlist-related functionality. It allows users to create playlists by entering a playlist name and selecting songs from the available music library.

Songs.js

The Songs.js component displays the list of available songs in the application. Users can view the songs and play them directly from the interface.

App.css

App.css defines the styling of the application. It controls the appearance of elements such as buttons, playlist sections, and song lists to create a clean and visually appealing interface.

index.js

The index.js file is the entry point of the React application. It loads the main App component and renders it inside the browser.

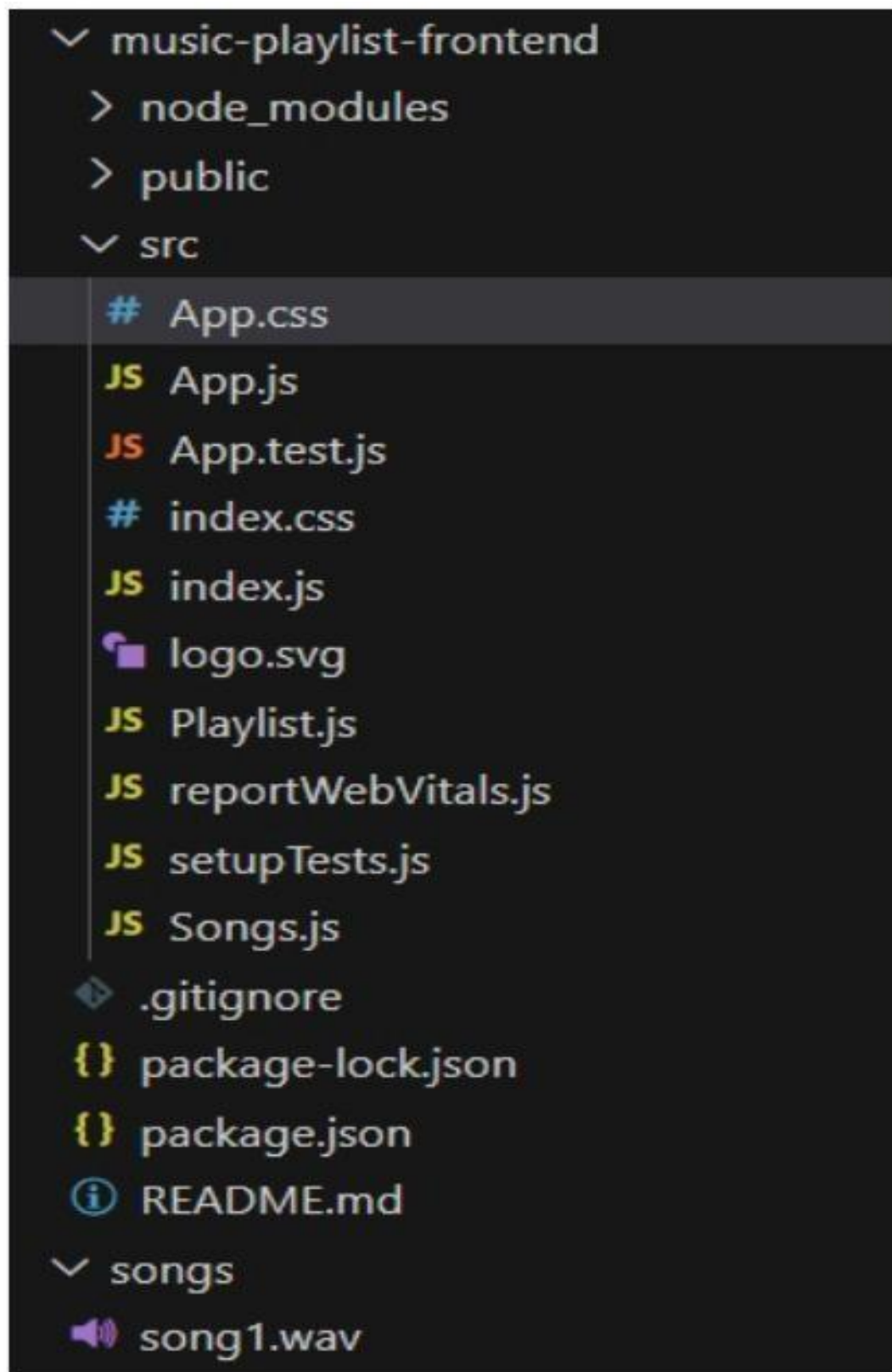


Figure 5.4: Frontend Project Folder Structure.

5.5 MongoDB Database Implementation

The Music Playlist Application uses **MongoDB** to store playlists and song information.

MongoDB stores data in collections and documents. In this project, a database named **musicdatabase** is used to store playlist data.

Each playlist document contains:

- Playlist name
- List of songs
- Unique playlist code

This structure allows users to organize songs into playlists and retrieve them efficiently.

Insert Screenshot

Place the FIFTH image here (MongoDB screenshot showing musicdatabase.playlists collection)

Figure 5.5: MongoDB Database Collection for Playlists.

5.6 Playlist Creation Functionality

One of the core features of the application is the ability to create playlists. Users can enter a playlist name and select songs from the available music library.

When the user clicks the Create Playlist button, the application sends a request to the backend server, which then stores the playlist information in the MongoDB database.

The system also generates a unique playlist code that can be used to share the playlist with collaborators.

This functionality demonstrates the interaction between the frontend interface, backend server, and database.

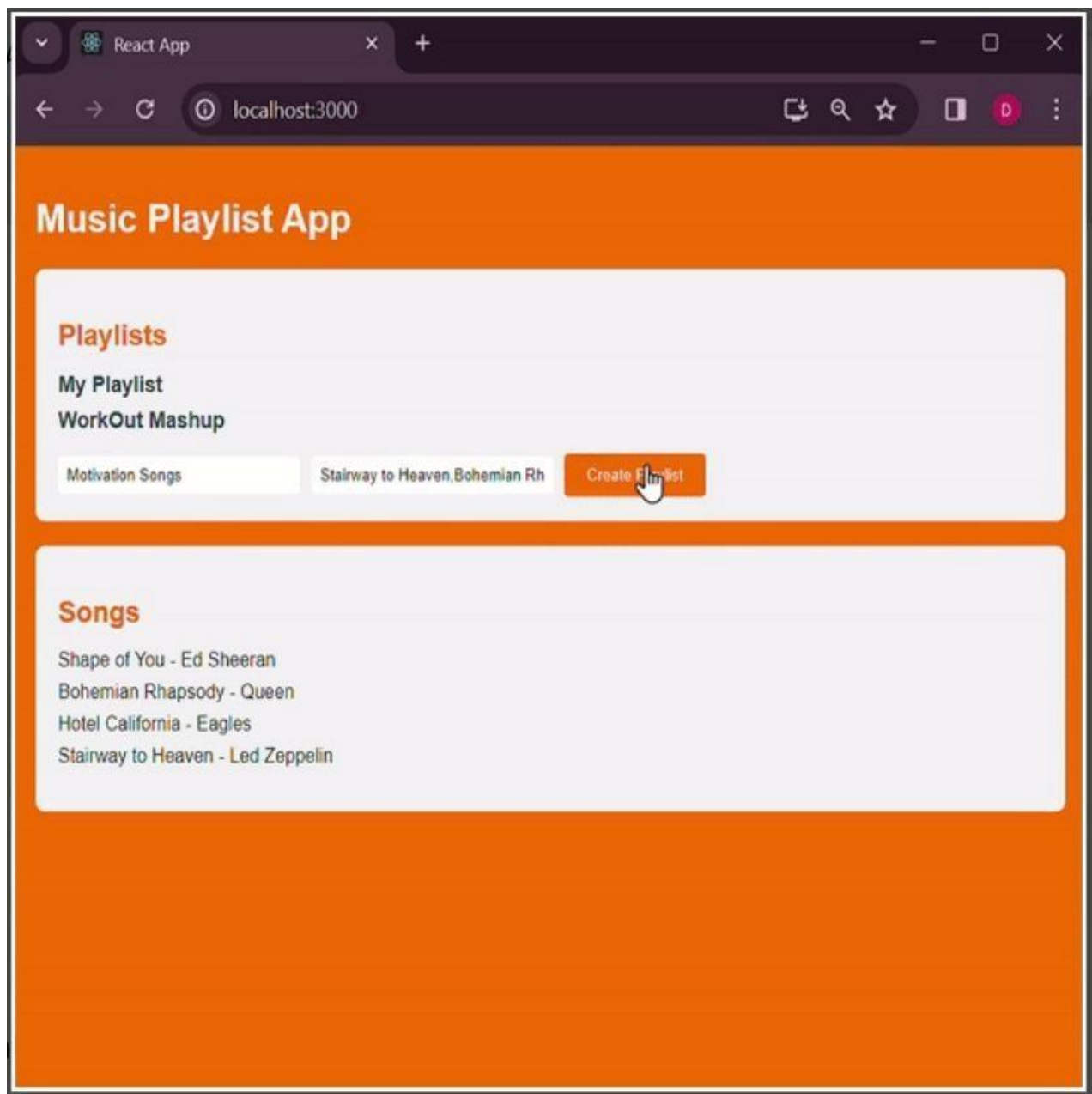


Figure 5.6: Playlist Creation Feature in the Application.

5.7 Song Playback Feature

The application allows users to play songs directly from the browser. When a user selects a song, the application streams the audio file from the server and plays it using the built-in audio player.

Playback controls include:

- Play
- Pause

- Next song
- Display currently playing song

This feature provides a seamless music listening experience.

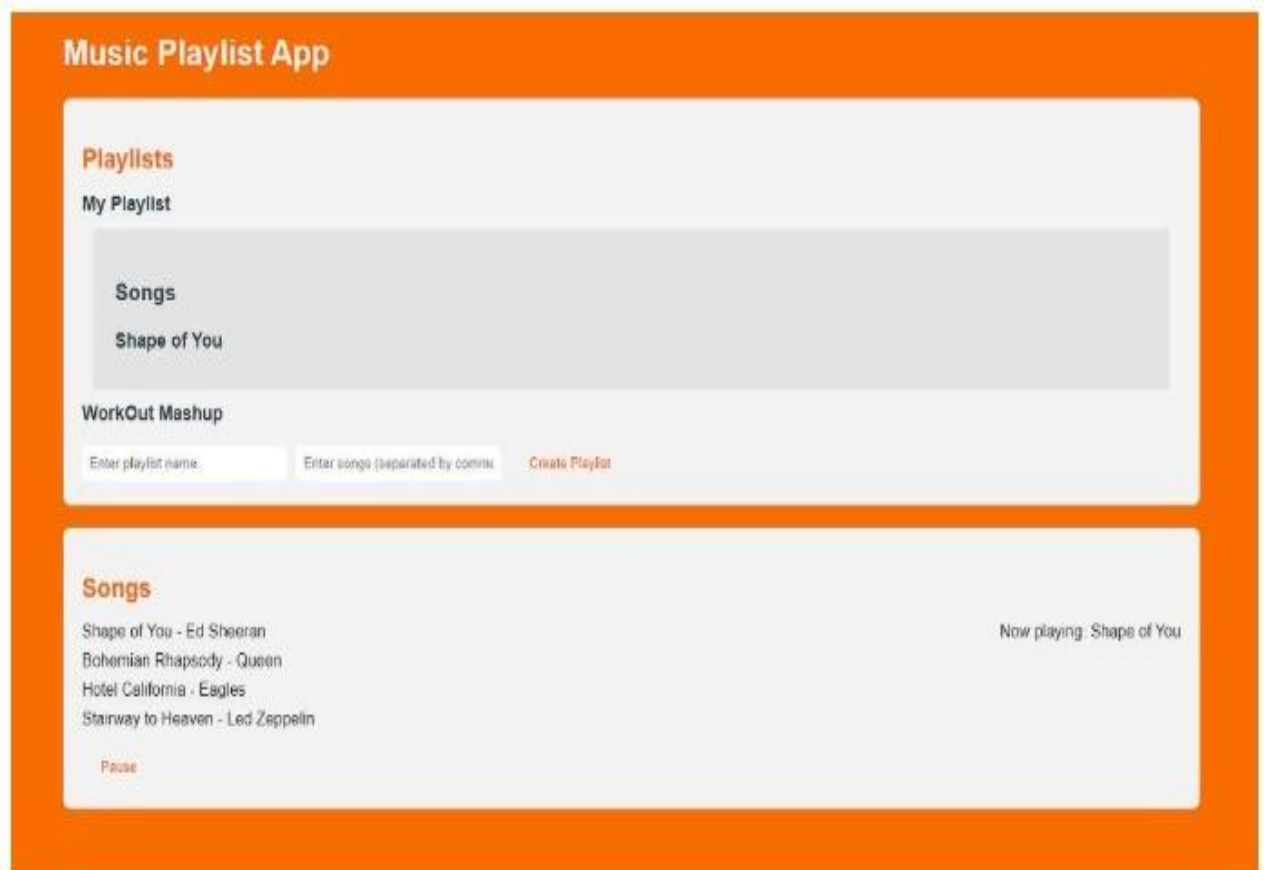


Figure 5.7: Song Playback Interface.

5.8 Integration of Frontend and Backend

The integration between the frontend and backend is achieved through REST APIs. When users perform operations such as creating playlists or retrieving songs, the frontend sends HTTP requests to the backend server.

The backend processes these requests and communicates with the MongoDB database to store or retrieve information.

The response is then sent back to the frontend and displayed to the user.

This integration ensures smooth communication between different layers of the application.

Chapter 6: Conclusion

This internship project successfully delivered a fully functional Music Playlist Application built on the MERN stack, demonstrating comprehensive full-stack development capabilities. The project journey encompassed the entire software development lifecycle, from initial requirements gathering through design, implementation, testing, and documentation.

Project Achievements:

The application provides users with intuitive tools for managing personal music collections. Key implemented features include playlist creation and management, song organization with comprehensive metadata, audio file upload and storage using GridFS, seamless audio streaming with playback controls, collaborative playlist capabilities for shared curation, and search and filtering functionality for easy discovery.

From a technical perspective, the project successfully implemented a RESTful API with Express.js handling all backend operations. The MongoDB database design leveraged document-oriented principles with GridFS for efficient audio file storage. The React frontend utilized component-based architecture with hooks for state management, providing responsive user interactions. Audio streaming was implemented through GridFS streams, enabling efficient delivery of audio content.

The development process followed industry best practices including modular code organization, environment-based configuration, comprehensive error handling, and thorough API testing. Version control was maintained throughout, documenting the evolution of the codebase.

REFERENCES

1. Banks, A., & Porcello, E. (2017). *Learning React: Functional Web Development with React and Redux*. O'Reilly Media.
2. Bradshaw, S., Brazil, E., & Chodorow, K. (2019). *MongoDB: The Definitive Guide (3rd ed.)*. O'Reilly Media.
3. Cantelon, M., Harter, M., Holowaychuk, T., & Rajlich, N. (2013). *Node.js in Action*. Manning Publications.
4. Dayley, B., & Dayley, B. (2014). *Node.js, MongoDB, and AngularJS Web Development*. Addison-Wesley Professional.
5. Flanagan, D. (2020). *JavaScript: The Definitive Guide (7th ed.)*. O'Reilly Media.
6. Freeman, E., Robson, E., Sierra, K., & Bates, B. (2004). *Head First Design Patterns*. O'Reilly Media.
7. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
8. Haverbeke, M. (2018). *Eloquent JavaScript (3rd ed.)*. No Starch Press.
9. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
10. Lawson, B., & Sharp, R. (2011). *Introducing HTML5*. New Riders Publishing.
11. Nixon, R. (2018). *Learning PHP, MySQL & JavaScript: With jQuery, CSS & HTML5 (5th ed.)*. O'Reilly Media.
12. Osmani, A. (2012). *Learning JavaScript Design Patterns*. O'Reilly Media.
13. Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media.
14. Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to Build HighPerformance Network Programs. *IEEE Internet Computing*, 14(6), 80